

Chap 1 - Introduction

Well-Posed Learning Problem

Definition

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

Example A checkers learning problem:

- Task T : playing checkers
- Performance Measure P : percent of games won against opponents
- Training Experience E : playing practice games against itself

Designing a learning program

1. Choose the training experience
2. Choose the target function
3. Choose a representation for the target function
4. Choose a learning algorithm to approximate the target function

Choice 1: the training experience

Feedback type

- direct (for example given a labeled board we are given the best move)
- indirect (only final game outcome is known (*credit assignment* which of many moves deserves credit for win))

Degree of Control - learner has over the sequence of training examples

Representation - How well the distribution of training examples represents the distribution over which performance P will ultimately be measured

Choice 2: The target function

Design a function that lets the program choose the best parameters for its goal. *Note* it is easier to learn an *evaluation function* than to learn how to "play" directly.

Choice 3: The representation

Here we have to learn an **approximation** because it is to computationally expensive to learn *true* function.

Choice 4: The learning algorithm

Estimate the *value, cost, error* of the current parameters to the true values and adjust accordingly.

Algorithm: LMS

Known as Least Mean Squares, fit the weights so that our approximation best matches the true values.

$$E = \sum_{\langle b, V_{train}(b) \rangle} (V_{train}(b) - \hat{V}(b))^2$$

Intuitively, when our approximation under-estimates the weights are nudged up in proportion to its feature's contribution, and vice-versa.

Final Design - Four Modules

1. Performance System - solves the task
2. Critic - Analyzes the performance system and produces training examples
3. Generalizer - Runs the learning algorithm to update approximation
4. Experiment Generator - Proposes the next problem to improve coverage

Perspectives

Learning, is defined as a search through candidate hypotheses for the one that best fits the training examples and constraints. These *learning algorithms* differ in:

- Hypothesis space being searched (decision trees, neural networks)
- Search strategy used (greedy, gradient descent)

Chap 2 - Concept Learning

Definition Concept Learning: inferring a boolean-valued function from training examples of its input and output

We want to learn a target concept(classification) from training examples, by choosing hypotheses from hypothesis space $\mathcal{H}$.

Hypothesis Representation: a conjunction of constraints on attributes. Each is either,

- A specific value (water=warm)
- " $\neq$ " - any value is acceptable
- " $=\emptyset$ " - no value is acceptable

Definition Inductive Learning Hypothesis: Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over other unobserved examples.

Fact General-to-specific Ordering: For hypotheses h_j, h_k defined over $\mathcal{X}$: h_j is **more-general-than-or-equal-to** h_k (written $h_j \geq_g h_k$) iff

$$\forall x \in \mathcal{X} : [(h_k(x) = 1) \rightarrow (h_j(x) = 1)] .$$

Find-S

Finding a maximally specific hypothesis

FIND-S Algorithm

1. Initialize h to the most specific hypothesis in $\mathcal{H}$.
2. For each positive training instance x :
 - For each attribute constraint a_i in h :
 - If a_i is not satisfied by x , replace a_i with the next more general constraint satisfied by x .
3. Output h .

Intuition: FIND-S ignores negative examples entirely and simply climbs the general-to-specific ordering, generalizing just enough at each positive example to keep covering it

Questions: Does it converge, why prefer maximally specific hypothesis (if not many) if there are others that are equally consistent, what if data is noisy

Version Spaces and Candidate Elimination

Fact A hypothesis h is **consistent** with a set of training examples D iff $h(x) = c(x)$ for every $\langle x, c(x) \rangle \in D$.

The **version space**, $VS_{H,D}$ is the subset of hypotheses from $\mathcal{H}$ consistent with the training examples in D :

$$VS_{H,D} \equiv \{h \in \mathcal{H} \mid \text{Consistent}(h, D)\}$$

Note: List-then-eliminate could work correctly, but it is intractable

Solution: Every member of the (possibly huge) version space is bracketed between two small *boundary sets*:

- G - the set of maximally general members of $VS_{H,D}$
- S - the set of maximally specific members of $VS_{H,D}$

$$VS_{H,D} = \{hH \mid \exists s \in S, \exists g \in G, g \geq_g h \geq_g s\}$$

S and G implicitly represent every hypothesis in the version space, without listing them.

Candidate-Elimination

Initialize:

$$G \leftarrow \{\text{most general hypothesis in } \mathcal{H}\}, \quad S \leftarrow \{\text{most specific hypothesis in } \mathcal{H}\}.$$

- 1: **for all** training examples d **do**
- 2: **if** d is a positive example **then**
- 3: Remove from G any hypothesis inconsistent with d .
- 4: **for all** $s \in S$ inconsistent with d **do**
- 5: Remove s from S .
- 6: Add to S all minimal generalizations h of s such that:
 h is consistent with d , and some member of G is more general than h .
- 7: Remove from S any hypothesis more general than another in S .
- 8: **end for**
- 9: **else if** d is a negative example **then**
- 10: Remove from S any hypothesis inconsistent with d .
- 11: **for all** $g \in G$ inconsistent with d **do**
- 12: Remove g from G .
- 13: Add to G all minimal specializations h of g such that:
 h is consistent with d , and some member of S is more specific than h .
- 14: Remove from G any hypothesis less general than another in G .
- 15: **end for**
- 16: **end if**
- 17: **end for**

Positive examples generalize S (and prune G)
negative examples specialize G (and prune S). The algorithm converges when $S = G = \{h\}$ which means a unique consistent hypothesis) or $S = G = \emptyset$ which means no consistent hypothesis in *mathcal{H}*.

Picking new training examples An active learner should query for the classification of the instance that best discriminates among remaining hypotheses. What-ever one will shrink version space by up to half.

Classifying new instances with a partially learned concept An instance can be classified confidently once every hypothesis in the version space agrees on it, a vote decides otherwise

Inductive Bias

To fix the fact that constraints can mean the learner cannot represent some concepts, which means Candidate-elimination does not converge:

Fix: an unbiased hypothesis space Let $\mathcal{H}'$ be the power set of $\mathcal{X}$ so every concept is representable.

An unbiased learned cannot generalize beyond the observed examples - this means some bias is needed for inductive generalization.

Consider a concept learning algorithm L for the set of instances $\mathcal{X}$. Let c be an arbitrary target concept,

$$D_C = \{ \langle x, c(x) \rangle \}$$

its training data, and $L(x_i, D_C)$ the classification L assigns to a new instance x_i after training on D_C .

The **inductive bias** of L is any minimal set of assertions B such that, for any target concept c and instances D_C ,

$$\forall x_i \in \mathcal{X} : \quad (B \wedge D_C \wedge x_i) \vdash L(x_i, D_C).$$

In short: The bias is the extra assumption that allows deductive inference to reproduce the learner's inductive prediction.

Note: Stronger bias means more generalization from less data, but greater risk of ruling out the true concept if bias is wrong.

Chap 3 - Probabilistic Machine Learning

Probabilistic ML: Treats unknown quantities as random variables with probability distributions. Useful for prediction and decision-making under uncertainty.

Supervised Learning

Learn a mapping

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

from inputs x to outputs y using training data

$$D_{\text{train}} = \{ \langle x_n, y_n \rangle \}_{n=1}^N .$$

- x : features / covariates / predictors

- y : label / target / response
 - **Classification**: y is discrete
 - **Regression**: y is continuous
- Classification**:

$$\mathcal{Y} = \{1, \dots, C\}$$

where C is the number of mutually exclusive classes. $C = 2$ is binary classification.

Design Matrix: For N examples and D features, $X \in \mathbb{R}^{N \times D}$. Rows = examples; columns = features.

Exploratory Data Analysis (EDA): Inspect data before modeling to identify patterns, relationships, outliers, and data-quality issues.

3.1.1 Empirical Risk Minimization

Misclassification Rate:

$$L(\theta) = \frac{1}{N} \sum_{n=1}^N \mathbb{I}[y_n \neq f(x_n; \theta)] .$$

Assumes all classification errors have equal cost.

General Loss:

$$\ell(y, \hat{y})$$

measures the cost of predicting $\hat{y}$ when the true output is y .

Empirical Risk:

$$L(\theta) = \frac{1}{N} \sum_{n=1}^N \ell(y_n, f(x_n; \theta)) .$$

ERM:

$$\hat{\theta} = \arg \min_{\theta} L(\theta) = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \ell(y_n, f(x_n; \theta))$$

Goal: Minimize expected loss on unseen data (**generalization**), not merely training loss.

3.1.2 Uncertainty

- **Epistemic uncertainty**: uncertainty from lack of knowledge about the true model; can decrease with more data.
- **Aleatoric uncertainty**: inherent randomness/noise in the data; generally irreducible.

For classification, model a conditional distribution:

$$p(y = c \mid x; \theta) = f_c(x; \theta),$$

where

$$0 \leq f_c(x; \theta) \leq 1, \quad \sum_{c=1}^C f_c(x; \theta) = 1.$$

3.1.3 Softmax Function

A model produces unnormalized scores called **logits**:

$$a = f(x; \theta) \in \mathbb{R}^C .$$

Softmax converts logits into class probabilities:

$$\text{softmax}(a)_c = \frac{e^{a_c}}{\sum_{j=1}^C e^{a_j}}$$

Thus,

$$p(y = c \mid x; \theta) = \text{softmax}(f(x; \theta))_c .$$

Properties:

$$0 \leq p(y = c \mid x; \theta) \leq 1, \quad \sum_{c=1}^C p(y = c \mid x; \theta) = 1.$$

Softmax maps

$$\mathbb{R}^C \rightarrow [0, 1]^C .$$

3.1.4 Logistic Regression

Special case where the function f is affine,

$$f(x; \theta) = b + w^T x, \text{ with } \theta = (w, b)$$

This is called logistic regression. With w = weights, b = bias. Bias gets absorbed into weights,

$$\hat{w} = [b, w_1, \dots, w_D], \hat{x} = [1, x_1, \dots, x_D] \text{so } \hat{w}^T \hat{x} = b + w^T x$$

So affine gets turned into linear.

3.1.5 Maximum Likelihood Estimation

Use negative log probability as the loss,

$$\uparrow(y, f(x; \theta)) = -\log p(y \mid f(x; \theta))$$

Negative log-likelihood (NLL)

$$NLL(\theta) = -\frac{1}{N} \sum_{n=1}^N \log p(y_n \mid f(x_n; \theta))$$

A good model assigns high probability to the true label, which means low NLL

3.1.6 Regression

Definition: Regression
Predict a real-valued output $y \in \mathbb{R}$
Quadratic Loss,

$$\uparrow_2(y - \hat{y}) = (y - \hat{y})^2$$

Mean Squared Error,

$$MSE(\theta) = \frac{1}{N} \sum_{n=1}^N (y_n - f(x_n; \theta))^2$$

Large residuals penalized more (use l_1 instead if outliers are a concern)

3.1.7 Gaussian Regression

Assume the target follows a Gaussian distribution:

$$y \mid x \sim \mathcal{N}(\mu(x), \sigma^2)$$

where $\mu(x) = f(x; \theta)$ is the model prediction.
Gaussian PDF:

$$\mathcal{N}(y \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

For fixed σ^2 , the negative log-likelihood is

$$NLL(\theta) = \frac{1}{2\sigma^2} MSE(\theta) + \text{const}$$

Therefore,

$$\text{MLE under Gaussian likelihood} \equiv \text{least-squares regression}$$

Key idea: Maximizing Gaussian likelihood is equivalent to minimizing squared error when variance is fixed.

3.1.8 Linear Regression

Simple Linear Regression (1D):

$$f(x; \theta) = b + wx$$

Multiple Linear Regression:

$$f(x; \theta) = b + w_1x_1 + \dots + w_Dx_D = b + w^T x$$

where b is the bias/intercept and w contains the feature weights.
Parameters are chosen to minimize mean squared error:

$$\hat{\theta} = \arg \min_{\theta} \text{MSE}(\theta)$$

Least Squares: Finds the parameters that minimize the sum of squared residuals (differences between predicted and true values).

$$\text{residual}_n = y_n - f(x_n; \theta)$$

Key connection:

$$\text{Linear Regression} + \text{Gaussian Noise} \implies \text{Least Squares} / \text{MSE}$$

3.1.9 Polynomial Regression & Feature Engineering

Polynomial regression transforms the input into polynomial features:

$$f(x; w) = w^T \phi(x), \quad \phi(x) = [1, x, x^2, \dots, x^D]^T$$

- Still **linear in the parameters** w , despite being nonlinear in x .
 - $\phi(x)$ is a **feature transformation**.
 - Manually choosing $\phi(x)$ is called **feature engineering**.
 - **Cross terms**, such as x_1x_2 , model interactions between features.
- Example 2D quadratic features:

$$\phi(x) = [1, x_1, x_2, x_1^2, x_2^2, x_1x_2]^T.$$

Key idea:

$$\text{nonlinear features} + \text{linear weights} \Rightarrow \text{polynomial regression.}$$

Degrees of Freedom and Overfitting

As polynomial degree $D \rightarrow N - 1$: one parameter per data point, leads to perfect interpolation MSE = 0 on training data. However, model overfits and predicts poorly on new inputs.

Transition to Deep Neural Networks
Learn the feature transform itself: let $\phi(x; V)$ have its own parameters V :

$$f(x; w, V) = w^T \phi(x; V)$$

Recursively compute:

$$f(x; \theta) = f_L(f_{L-1}(\dots f_1(X) \dots))$$

Final layer is linear, so the function itself is a learned feature extractor - core idea of deep neural networks & CNNs.

3.1.10 Empirical Risk vs. Population Risk

Empirical Risk: Average loss on the training data.

$$L(\theta; D_{\text{train}}) = \frac{1}{|D_{\text{train}}|} \sum_{(x,y) \in D_{\text{train}}} \ell(y, f(x; \theta))$$

Population Risk: Expected loss over the true data distribution p^* .

$$L(\theta; p^*) = \mathbb{E}_{(x,y) \sim p^*} [\ell(y, f(x; \theta))]$$

Generalization Gap:

$$L(\theta; p^*) - L(\theta; D_{\text{train}})$$

- Small gap $\Rightarrow$ model generalizes well.
- Large gap with low training risk $\Rightarrow$ **overfitting**.

Approach

Data is split into training, validation, and test sets. Training shouldn't be used in entirely as model would grow too complex and overfit, so we use validation sets to minimize risk, and a final test set to give an unbiased performance estimate.

Unsupervised Learning

Given only inputs $D_{\text{train}} = \{x_n\}_{n=1}^N$, fit an unconditional model $p(x)$ that can generate new data, vs supervised learning's conditional model $p(y \mid x)$

3.3.1 Clustering

Partition inputs into regions of “similar“ points

3.3.2 Factor Analysis & Latent Variables

Assume high-dimensional data $x_n \in \mathbb{R}^D$ is generated from lower-dimensional **latent variables** $z_n \in \mathbb{R}^K$, typically $K < D$.

$$z_n \rightarrow x_n$$

Factor Analysis (FA):

$$p(x_n \mid z_n; \theta) = \mathcal{N}(x_n \mid Wz_n + \mu, \Sigma)$$

- Similar to linear regression, but z_n is **unobserved**.
 - W : maps latent factors to observed features.
 - μ : mean; Σ : noise covariance.
 - If $\Sigma = \sigma^2 I \Rightarrow$ **Probabilistic PCA**.
 - Nonlinear mapping $f(z_n; \theta) \Rightarrow$ basis for **Variational Autoencoders (VAEs)**.
- Key idea:

$$\text{low-dimensional latent } z \longrightarrow \text{high-dimensional observed } x$$

3.3.3 Self-supervised Learning

Create proxy supervised tasks from unlabeled data, e.g. predict color from grayscale.

- Learns useful features without needing the “true“ latent factors z
- Features transferred to downstream supervised tasks

3.3.4 Evaluating Unsupervised Learning

Unsupervised learning is difficult to evaluate because there is often **no ground truth**.

1. **Density Estimation:** Evaluate negative log-likelihood (NLL) on held-out data:

$$L(\theta; \mathcal{D}) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \log p(x \mid \theta)$$

- Lower NLL $\Rightarrow$ model assigns higher probability to real data.
 - Can be difficult in high dimensions or affected by memorization.
- 2. Downstream Task Performance:** Evaluate whether the learned representation improves performance or **sample efficiency** on a supervised task.
- Other Goal:** Unsupervised learning may prioritize **interpretability** or scientific understanding rather than predictive performance.

Reinforcement Learning

An agent learns a policy $a = \pi(x)$ mapping states to actions by interacting with an environment and receiving occasional reward signals.

Preprocessing Discrete Inputs

One-Hot Encoding: Convert a K -valued categorical feature into a binary vector:

$$\text{onehot}(x) = [\mathbb{I}[x = 1], \dots, \mathbb{I}[x = K]]$$

Only the entry corresponding to the category is 1; all others are 0.

Feature Crosses: Add features representing combinations of categorical values to capture **interaction effects**.

$$\mathbb{I}[x_1 = k, \ x_2 = j]$$

- One-hot features capture individual category effects.
- Feature crosses capture interactions between categories.

Preprocessing Text: Bag of Words

Bag of Words (BoW): Represent a document by the number of times each vocabulary word occurs.

$$\tilde{x}_{nv} = \sum_{t=1}^T \mathbb{I}[x_{nt} = v]$$

where $\tilde{x}_{nv}$ is the count of word v in document n .

- Converts documents into vectors in $\mathbb{R}^D$.
- **Ignores word order.**
- Common preprocessing: lowercase, remove stop words, stemming.
- Collection of documents forms a **term-frequency matrix**.

TF-IDF & Word Embeddings

TF-IDF: Weights words by importance, downweighting common, uninformative words.

$$\text{IDF}_i = \log \frac{N}{1 + \text{DF}_i}$$

$$\text{TFIDF}_{ij} = \log(\text{TF}_{ij} + 1) \text{IDF}_i$$

- TF = frequency of word i in document j .
- DF = number of documents containing word i .
- High TF + low DF $\Rightarrow$ important word.

Word Embeddings: Map sparse one-hot vectors to low-dimensional dense vectors:

$$e = Ex, \quad E \in \mathbb{R}^{K \times V}$$

Similar words should have nearby embeddings.

Out-of-Vocabulary Words

OOV Problem: Test data may contain words not present in the training vocabulary.

- Simple solution: map unknown words to an **UNK** token.
- Better: split words into **subword units / wordpieces**.
- **Byte-Pair Encoding (BPE)** learns common subword units, allowing unseen words to be represented from known pieces.

Handling Missing Data

Let $M_{nd} = 1$ if feature d of example n is missing.

- **MCAR – Missing Completely At Random:** missingness is independent of all data.

$$p(M \mid X, Y) = p(M)$$

- **MAR – Missing At Random:** missingness depends only on **observed** values.
- **NMAR – Not Missing At Random:** missingness depends on the **missing value itself**; the missingness mechanism must be modeled.

Imputation: Replace missing values with estimates. Simple approach: **mean imputation**; more advanced approaches model the missing values probabilistically.